Distributed Task Management System with Real-Time Collaboration

Design Document

C-DAC, Bangalore

Date: 10 July 2026

Prepared By:
Harsh Shivankar (260250120057)
Anshu Raj Singh (260250120021)
Nilesh Jaiswal (260250120087)
Ayush Sharma (260250120034)
Saurabh Kumar Yadav (260250120121)

Project Code	Project 6 - PG-DAC 2026
Date	10 July 2026
Status	Approved By Mentor
Prepared by	Harsh Shivankar, Anshu Raj Singh, Nilesh Jaiswal, Ayush Sharma, Saurabh Kumar Yadav

1. Introduction

Remote and distributed teams frequently struggle to find a unified tool that combines task management, live multi-user collaboration, conflict detection, and role-based access control in one platform. This project, Distributed Task Management System with Real-Time Collaboration (TaskFlow), addresses that gap by building a web-based Kanban board where team members can create, assign, update, and track tasks, with a full activity audit trail.

1.1 Purpose

The purpose of this document is to give a detailed design for the Distributed Task Management System. It explains the architecture of the system, the database design, the component design, and the API design. This document is primarily for developers to understand how the system is built and how all the parts connect to each other.

1.2 Scope

This design document covers the backend (Java Spring Boot), the database (PostgreSQL), and the API layer of the TaskFlow system. The React frontend design is partially covered in the Interface Design section. Real-time WebSocket collaboration is designed but implementation is in progress.

1.3 Intended Audience

This document is intended to be read by all team members, the project mentor, and evaluators at CDAC-ACTS. It should be referred to during development and evaluation.

1.4 References

- Software Requirement Specification - Distributed Task Management System v1.0 (10 June 2026)
- Spring Boot 3.x Documentation: https://spring.io/projects/spring-boot
- PostgreSQL 15 Documentation: https://www.postgresql.org/docs/15/
- STOMP over WebSocket Protocol: https://stomp.github.io/
- React 18 Documentation: https://react.dev

2. Acronyms, Terms and Definitions

Term	Definition
SRS	Software Requirement Specification
API	Application Programming Interface - RESTful HTTP endpoints exposed by the backend
JWT	JSON Web Token - stateless token issued after login, sent in Authorization header
WebSocket	Full-duplex communication protocol over TCP used for real-time updates
STOMP	Simple Text Oriented Messaging Protocol used over WebSocket for structured messaging
RBAC	Role Based Access Control - different users have different permissions
JPA	Java Persistence API - used to interact with the database using Java objects
ORM	Object Relational Mapping - maps Java classes to database tables
BCrypt	Password hashing algorithm used to safely store passwords
JSONB	JSON Binary - PostgreSQL column type that stores JSON and allows querying inside it
DTO	Data Transfer Object - plain Java class used to send/receive data in API requests
REST	Representational State Transfer - style of building APIs using HTTP methods
CRUD	Create, Read, Update, Delete - the four basic database operations
PG-DAC	Post Graduate Diploma in Advanced Computing

3. Assumptions and Constraints

Assumptions:

- PostgreSQL database is already installed and running on localhost port 5432 before starting the application.
- The Spring Boot application and the database are running on the same machine during evaluation.
- All team members have Java 22 and Maven installed on their machines.
- JWT secret key and database credentials are stored in application.properties for the academic submission. In a real production system these would be in environment variables.
- Users will access the application using a modern browser like Chrome or Firefox.

Constraints:

- This system is built for academic demonstration. It is not production-ready (no HTTPS, no Docker, no cloud deployment).
- Real-time WebSocket module is designed but full end-to-end testing with the React frontend is pending.
- Email notifications are out of scope for mid evaluation. Only in-app notifications are implemented.
- File attachments on tasks are out of scope.
- Mobile app (iOS/Android) is out of scope.

4. Basic Design Approach

The TaskFlow system follows a standard three-layer architecture:

Layer	Technology	What it does
Presentation	React 18 + Tailwind CSS	What the user sees and interacts with in the browser
Business Logic	Java Spring Boot 3.3	All the rules and processing - controllers, services, security
Data	PostgreSQL 15	Stores all data permanently in tables

Within the backend, we follow the Controller → Service → Repository pattern:

- Controller: receives the HTTP request, checks input, calls the service, returns the response.
- Service: contains all the business logic. Calls the repository. Also calls AuditLogService and NotificationService when needed.
- Repository: Spring Data JPA interface that talks to the PostgreSQL database.

All API errors are handled by a single GlobalExceptionHandler class which maps exceptions to the right HTTP status codes and returns a JSON error body. This means every error response in the system has the same format: { "error": "message" }.

5. Risks

No	Risk	Impact	Mitigation
1	Real-time WebSocket module not fully tested before final evaluation	Person 4 demo may not work live	Code is written and designed. Will be completed and tested before final eval.
2	PostgreSQL not set up correctly on demo machine	App will not start	Test on the demo machine at least one day before evaluation.
3	Optimistic lock version mismatch if tester does not read the response version	409 conflict error during demo	Document the version field in test script and always use the version from the last response.
4	React frontend not integrated with backend by final eval	Cannot show full end-to-end flow	Backend APIs are fully working and tested with Postman. Frontend integration is in progress.
5	Port 8080 already in use on demo machine	App will not start	Kill the process using the port or change port in application.properties.

6. System Overview

TaskFlow is a web-based Kanban task management system. Users register and log in to get a JWT token. With that token they can create projects, invite other team members, and manage tasks on a Kanban board with columns like To Do, In Progress, and Done.

Every action on a task (create, update, move) is automatically recorded in an audit log. Task assignees get in-app notifications. An analytics API gives a summary of task counts by column, overdue tasks, and tasks per member.

The real-time module (in progress) will use Spring WebSocket with STOMP so that when one user moves a task, all other users watching the same board will see it move instantly without refreshing the page.

The system has five main modules:

- Authentication Module - register, login, JWT, refresh token
- Project Management Module - create projects, add members, view board
- Task Management Module - create, update, move tasks, optimistic locking
- Real-Time Collaboration Module - WebSocket broadcast of board events (in progress)
- Audit Log, Notifications and Analytics Module - audit trail, notifications, analytics

7. Architecture Design

7.1 Overall Architecture

The system uses a standard client-server architecture with three tiers. The React frontend talks to the Spring Boot backend using HTTP REST APIs and WebSocket. The Spring Boot backend talks to the PostgreSQL database using Spring Data JPA.

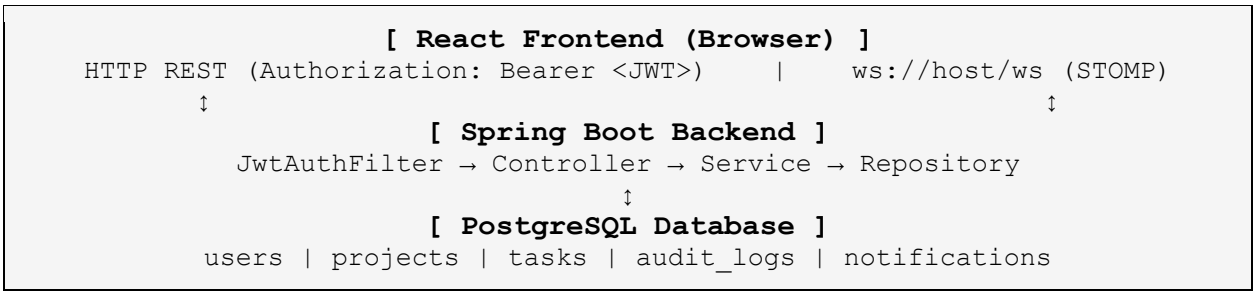

Figure 1: System Architecture Diagram (text representation)

7.2 Technology Stack

Layer	Technology	Reason for choosing
Frontend	React 18 + Tailwind CSS	Component based UI, easy to build dynamic boards
Backend	Java 22 + Spring Boot 3.3	Industry standard, strong security support, easy REST + WebSocket
Database	PostgreSQL 15	ACID compliant, supports JSONB for audit log values
Security	Spring Security + JWT (JJWT)	Stateless authentication, no session storage needed on server
ORM	Spring Data JPA + Hibernate	Maps Java classes to DB tables, handles optimistic locking via @Version
Real-Time	Spring WebSocket + STOMP	Built into Spring, works with Spring Security, named topic subscriptions
Build Tool	Maven	Standard Java build tool, easy dependency management

7.3 Security Design

Every request to a protected endpoint goes through JwtAuthFilter before reaching the controller. The filter reads the Authorization: Bearer <token> header, validates the token using JwtUtil, loads the user from the database, and sets the authentication in Spring Security's context. If the token is missing or invalid, the request is rejected with HTTP 401 before any controller code runs.

The request flow for a protected API call is:

- Client sends HTTP request with Authorization: Bearer <token> header
- JwtAuthFilter intercepts the request
- JwtUtil validates the token (signature, expiry)
- UserDetailsServiceImpl loads the user from the database
- Spring Security sets the authentication context
- Request reaches the Controller
- Controller calls Service with the authenticated user
- Response sent back to client

8. Data Design

All data is stored in a PostgreSQL 15 database named taskflow_db. The schema is automatically created and updated by Hibernate (spring.jpa.hibernate.ddl-auto=update) when the Spring Boot application starts.

8.1 Database Tables

Table: users

Stores all registered users. Password is never stored in plain text - only the BCrypt hash.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID for each user
-	**name**	VARCHAR	Full name of the user
-	**email**	VARCHAR	Email address - must be unique across all users
-	**password_hash**	VARCHAR	BCrypt hashed password - never the plain text
-	**role**	VARCHAR	System role: ADMIN, MANAGER, MEMBER, or VIEWER
-	**created_at**	TIMESTAMP	When the user registered

Table: projects

Stores all projects created in the system.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**owner_id**	BIGINT	References users.id - the person who created the project
-	**name**	VARCHAR	Name of the project
-	**description**	TEXT	Optional description
-	**archived**	BOOLEAN	True if the project is archived (read-only)
-	**created_at**	TIMESTAMP	When the project was created

Table: project_members

Join table connecting users to projects with a role for that project.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**project_id**	BIGINT	References projects.id
FK	**user_id**	BIGINT	References users.id
-	**role**	VARCHAR	Role in this project: MANAGER, MEMBER, or VIEWER

Table: board_columns

Stores the Kanban columns for each project. Default columns are To Do, In Progress, Done.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**project_id**	BIGINT	References projects.id
-	**name**	VARCHAR	Column name e.g. To Do, In Progress, Done
-	**position**	INT	Order of the column on the board (1, 2, 3...)

Table: tasks

Stores all tasks. The version column is used for optimistic locking - JPA increments it on every update.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**project_id**	BIGINT	References projects.id
FK	**column_id**	BIGINT	References board_columns.id - current column of the task
FK	**assignee_id**	BIGINT	References users.id - who the task is assigned to (nullable)
-	**title**	VARCHAR	Title of the task (required)
-	**description**	TEXT	Detailed description (optional)
-	**due_date**	TIMESTAMP	When the task is due (optional)
-	**priority**	VARCHAR	LOW, MEDIUM, HIGH, or CRITICAL
-	**version**	BIGINT	Optimistic lock version - starts at 0, incremented on every save
-	**created_at**	TIMESTAMP	When the task was created

Table: audit_logs

Immutable record of every important action in the system. Rows are only ever inserted, never updated or deleted.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**actor_id**	BIGINT	References users.id - who performed the action
FK	**project_id**	BIGINT	References projects.id - which project this event belongs to

-	**event_type**	VARCHAR	What happened: TASK_CREATED, TASK_MOVED, TASK_UPDATED etc.
-	**entity_type**	VARCHAR	What kind of thing changed: TASK, PROJECT etc.
-	**entity_id**	BIGINT	ID of the thing that changed
-	**old_value**	TEXT	JSON string of the values before the change (null for create events)
-	**new_value**	TEXT	JSON string of the values after the change
-	**created_at**	TIMESTAMP	Exactly when this event happened

Table: notifications

In-app notifications for users. Created automatically when tasks are assigned or moved.

Key	Field Name	Data Type	Description
PK	**id**	BIGINT	Auto-generated unique ID
FK	**user_id**	BIGINT	References users.id - who this notification is for
-	**message**	TEXT	The notification message text
-	**read**	BOOLEAN	False when first created, set to true when user marks as read
-	**created_at**	TIMESTAMP	When the notification was created

9. Component Design

The backend is divided into the following components. Each component follows the Controller - Service - Repository pattern.

9.1 Authentication Component

Responsible: Harsh Shivankar

Handles user registration, login, and token management. This is the entry point for all users before they can access any other part of the system.

Key files:

- AuthController.java - REST endpoints for register, login, refresh
- AuthService.java - business logic for auth operations
- JwtUtil.java - generates and validates JWT tokens
- JwtAuthFilter.java - Spring Security filter that checks JWT on every request
- UserDetailsServiceImpl.java - loads user from DB for Spring Security
- GlobalExceptionHandler.java - maps all exceptions to JSON responses

Step by step flow:

- Step 1: User sends POST /api/v1/auth/register with name, email, password
- Step 2: AuthController calls AuthService.registerFull()
- Step 3: AuthService checks if email already exists - if yes throws ConflictException (409)
- Step 4: Password is hashed with BCrypt and User is saved to the users table
- Step 5: A refresh token is generated and set as an HTTP-only cookie
- Step 6: Response: HTTP 201 with { message, userId }

9.2 Project Management Component

Responsible: Anshu Raj Singh

Handles creating projects, listing projects, adding members, and getting the board view. When a project is created, three default columns are automatically seeded.

Key files:

- ProjectController.java - REST endpoints for project operations
- ProjectService.java - business logic for projects and board
- Project.java, ProjectMember.java, BoardColumn.java - JPA entities
- ProjectRepository.java, ProjectMemberRepository.java, BoardColumnRepository.java

Step by step flow:

- Step 1: User sends POST /api/v1/projects with name and description
- Step 2: JwtAuthFilter validates the token and sets the authenticated user
- Step 3: ProjectController gets the User entity and calls ProjectService.createProject()

- Step 4: ProjectService creates the Project, saves it
- Step 5: ProjectService creates 3 BoardColumn entries: To Do (pos 1), In Progress (pos 2), Done (pos 3)
- Step 6: ProjectService adds the creator as a MANAGER in project_members
- Step 7: Response: HTTP 201 with project details

9.3 Task Management Component

Responsible: Nilesh Jaiswal

Handles creating, updating, and moving tasks. Uses JPA optimistic locking via the @Version annotation on the Task entity to prevent two users from overwriting each other's changes.

Key files:

- TaskController.java - REST endpoints for task operations
- TaskService.java - business logic including optimistic lock check
- Task.java - JPA entity with @Version field
- TaskRepository.java

Step by step flow:

- Step 1: User sends PATCH /api/v1/tasks/{id}/move with { columnId, version }
- Step 2: TaskService loads the task from database - current version is e.g. 2
- Step 3: TaskService checks: does the version in the request match version in database?
- Step 4: If version matches: task column is updated, version becomes 3, audit log is written, WebSocket broadcast is sent
- Step 5: If version does not match: ConflictException is thrown - response is HTTP 409 with { error, currentVersion }
- Step 6: This prevents the case where User A and User B both read version 2, User A saves first making it version 3, and then User B tries to save version 2 overwriting User A's work

9.4 Real-Time Collaboration Component

Responsible: Saurabh Kumar Yadav

Provides real-time updates to all users viewing the same board using Spring WebSocket and STOMP protocol. When a task is moved or created, all connected clients receive an event automatically. This component is in progress.

Key files:

- WebSocketConfig.java - configures STOMP broker and /ws endpoint
- WebSocketAuthInterceptor.java - extracts JWT during WebSocket handshake
- PresenceController.java - broadcasts USER_JOINED when user connects
- WebSocketMessage.java - DTO for all WebSocket events

Step by step flow:

- Step 1: Client connects to ws://localhost:8080/ws/websocket?token=<JWT>
- Step 2: WebSocketAuthInterceptor validates the JWT during the handshake
- Step 3: Client sends STOMP SUBSCRIBE to /topic/project/1
- Step 4: Spring fires SessionSubscribeEvent - PresenceController broadcasts USER_JOINED to /topic/project/1
- Step 5: When any user calls PATCH /tasks/{id}/move, TaskService calls broadcast()
- Step 6: broadcast() sends a TASK_MOVED message to /topic/project/1
- Step 7: All subscribed clients receive the message and can update their UI

9.5 Audit Log, Notifications and Analytics Component

Responsible: Ayush Sharma

Tracks every important action in the system and stores it in the audit_logs table. Also manages in-app notifications and provides analytics data for the project dashboard.

Key files:

- AuditLogService.java - single method log() called from TaskService
- AuditLogController.java - GET /api/v1/projects/{id}/audit
- NotificationService.java - saves and pushes notifications
- NotificationController.java - GET /api/v1/users/me/notifications
- AnalyticsService.java - computes task counts, overdue, per-member
- AnalyticsController.java - GET /api/v1/projects/{id}/analytics
- AuditLog.java, Notification.java - JPA entities

Step by step flow:

- Step 1: TaskService creates a task and calls AuditLogService.log(actor, project, TASK_CREATED, TASK, taskId, null, newValue)
- Step 2: AuditLogService creates an AuditLog entry and saves it - this is a pure INSERT, never UPDATE or DELETE
- Step 3: If the task has an assignee, TaskService calls NotificationService.notify(assignee, message)
- Step 4: NotificationService saves a Notification entry to the notifications table
- Step 5: NotificationService also pushes the notification to /queue/notifications for the user via WebSocket
- Step 6: User calls GET /api/v1/users/me/notifications to see their notifications
- Step 7: Manager calls GET /api/v1/projects/1/analytics to get task counts and workload data

10. API Design

All APIs are under the base path /api/v1. All endpoints except /auth/** require an Authorization: Bearer <JWT> header. Errors always return JSON in the format { "error": "message" }.

Method	Endpoint	Auth	Request Body	Response
POST	/api/v1/auth/register	No	{ name, email, password }	201: { message, userId } | 409: email exists
POST	/api/v1/auth/login	No	{ email, password }	200: { accessToken, tokenType } | 401: wrong password
POST	/api/v1/auth/refresh	Cookie	(refresh token in HTTP-only cookie)	200: { accessToken } | 401: token expired
GET	/api/v1/projects	Yes	(none)	200: array of projects for the logged in user
POST	/api/v1/projects	Yes	{ name, description }	201: project object with id
POST	/api/v1/projects/{id}/members	Yes	{ userId, role }	200: { message } | 409: already a member
GET	/api/v1/projects/{id}/board	Yes	(none)	200: board with columns and tasks
POST	/api/v1/projects/{id}/tasks	Yes	{ title, columnId, priority, assigneeId, dueDate }	201: task object with version: 0
PATCH	/api/v1/tasks/{id}	Yes	{ title?, description?, priority?, version }	200: updated task | 409: stale version
PATCH	/api/v1/tasks/{id}/move	Yes	{ columnId, version }	200: moved task | 409: stale version with currentVersion
GET	/api/v1/projects/{id}/audit	Yes	?page=0&size=20	200: { page, totalEvents, events[] }
GET	/api/v1/projects/{id}/analytics	Yes	(none)	200: { taskCountByStatus, overdueTaskCount, tasksPerMember }
GET	/api/v1/users/me/notifications	Yes	(none)	200: array of unread notifications
PATCH	/api/v1/users/me/notifications/read	Yes	(none)	204: all marked as read

11. Interface Design

The React frontend is currently in progress. Below are the main screens planned for the application.

Screen	Description
Login / Register Page	User enters email and password. On success, JWT token is stored in memory and refresh token is set as cookie. Redirects to the project list page.
Project List Page	Shows all projects the user is a member of. Has a button to create a new project. Each project card shows the name and a link to open the board.
Kanban Board Page	Shows all columns side by side. Each column has task cards. User can move tasks between columns using drag and drop. Board updates in real time when other users make changes (in progress).
Task Detail Panel	Opens as a side panel when a task card is clicked. Shows title, description, assignee, due date, priority. User can edit fields and save. Shows the current version number.
Audit Log Page	Shows a table of all actions on the project. Each row shows who did what and when. Managers can filter by event type.
Analytics Page	Shows task count per column, overdue task count, and a breakdown of tasks per team member.
Notifications Bell	Icon in the top navigation bar. Shows count of unread notifications. Clicking opens a dropdown list of notifications.

12. Specific Design Considerations

- Optimistic Locking: We chose JPA @Version based optimistic locking instead of database-level pessimistic locks. This means the database is never locked waiting for a user to finish editing. The conflict is detected only at save time and the second user is informed with the current version in the error response.
- Single Audit Call Site: AuditLogService.log() is called from TaskService only. This means the audit log format is always consistent and we cannot forget to log something because all task changes go through TaskService.
- GlobalExceptionHandler: All exception handling is in one place. Adding a new exception type only requires adding one method in this class. Every error response in the entire system has the same shape.
- HTTP-only Refresh Token Cookie: The refresh token is stored in an HTTP-only cookie so JavaScript cannot read it. This protects against XSS attacks stealing the refresh token.
- API Versioning: All endpoints are under /api/v1. If we need to make breaking changes in the future we can add /api/v2 without affecting existing clients.
- JSONB Audit Values: Old and new values in the audit log are stored as JSON strings in a TEXT column. PostgreSQL can query inside these as JSONB which lets us do queries like 'show me all tasks that moved to column 3'.

13. Design Test

All APIs have been manually tested using Postman. The following types of tests were done:

Test Type	What was tested	Result
Happy path	Register, login, create project, create task, move task in order	All return correct HTTP status and response body
Auth failure	Call protected API without token, with wrong token, with expired token	All return 401 with JSON error
Duplicate email	Register with same email twice	Second request returns 409 conflict
Wrong password	Login with correct email but wrong password	Returns 401 with 'Invalid credentials'
Optimistic lock	Move task with stale version number	Returns 409 with currentVersion in response
Audit log	Create and move tasks then check audit log API	All events recorded correctly with old and new values
Analytics	Create tasks in different columns then check analytics API	Returns correct counts per column

14. Cross Reference with SRS

ID	Feature	API / Component	Status
F01	User Auth & RBAC	/api/v1/auth/register, /api/v1/auth/login, /api/v1/auth/refresh, JWT filter	Complete - register, login, refresh token, JWT auth all working. Role hierarchy enforcement is pending.
F02	Project Management	/api/v1/projects, /api/v1/projects/{id}/members, /api/v1/projects/{id}/board	Complete - create, list, add members, board view all working. Archive and delete project pending.
F03	Kanban Board	/api/v1/projects/{id}/tasks, /api/v1/tasks/{id}/move	Complete - create task, move task working. Frontend drag-and-drop pending.
F04	Real-Time Sync	WebSocket /ws, /topic/project/{id}	Partially complete - WebSocket config and PresenceController written. End-to-end testing with frontend pending.
F05	Conflict Detection	PATCH /api/v1/tasks/{id} and /move with version field	Complete - optimistic locking via @Version working. 409 response with currentVersion confirmed.
F06	Deadline Engine	DeadlineScheduler.java (@Scheduled)	Partially complete - scheduler class exists and queries overdue tasks. Email notification delivery pending.
F07	Activity Audit Log	GET /api/v1/projects/{id}/audit	Complete - all task events recorded. Date range filtering pending.
F08	Notifications	GET /api/v1/users/me/notifications, NotificationService	Complete for in-app. Email notifications pending.
F09	Analytics Dashboard	GET /api/v1/projects/{id}/analytics	Complete for backend API. React charts pending.
F10	Admin Panel	/api/v1/admin/**	Not yet started. Planned for final evaluation.

— End of Document —